Team Note of ICPCTTALKAK

구윤우, 정민건, 황하율

Compiled on November 13, 2026

Contents

1 자료 구조 **2**
- 1.1 펜윅 트리(Fenwick Tree / BIT) + 예제 . . . 2
- 1.2 bitset 사용법 + 부분집합 합 예제 . . . 2
- 1.3 unordered_set / unordered_map 사용자 정의 해시 . . . 2
- 1.4 세그먼트 트리 . . . 3
- 1.5 세그먼트 트리 + 레이지 프로퍼게이션 . . . 3
- 1.6 퍼시스턴트 세그먼트 트리 . . . 3

2 그래프/트리 **3**
- 2.1 단절점 및 단절선 . . . 3
- 2.2 SCC - Kosaraju . . . 4
- 2.3 BCC - Tarjan . . . 6
- 2.4 최대 유량 - Dinic . . . 6
- 2.5 MCMF . . . 7
- 2.6 이분 매칭 - Hopcroft Karp . . . 7
- 2.7 최소 공통 조상(LCA) . . . 8
- 2.8 Heavy Light Decomposition . . . 8
- 2.9 센트로이드 . . . 9

3 동적계획법 **9**
- 3.1 Convex Hull Trick . . . 9
- 3.2 aliens trick + monotone queue optimization . . . 10
- 3.3 Dnc optimization . . . 11

4 수학 **11**
- 4.1 나눗셈, 최대공약수, 최소공배수 . . . 11
- 4.2 빠른 거듭제곱 . . . 11
- 4.3 소수 판별, 소인수분해 . . . 11
- 4.4 에라토스테네스의 체, 소인수분해 . . . 11
- 4.5 선형 시간 체, 곱셈적 함수 전처리 . . . 12
- 4.6 확장 유클리드 알고리즘 . . . 12
- 4.7 중국인의 나머지 정리 . . . 12
- 4.8 이항 계수를 소수로 나눈 나머지 . . . 12
- 4.9 빠른 소수 판별, 소인수분해 - Miller Rabin, Pollard Rho . . . 12
- 4.10 가우스 소거법 - RREF, 랭크, 행렬식, 역행렬 . . . 13
- 4.11 다항식 곱셈(FFT) . . . 14
- 4.12 뫼비우스 함수 . . . 14

5 문자열 **15**
- 5.1 문자열 해싱(Rolling Hash) + 예제 . . . 15
- 5.2 문자열 매칭 - KMP . . . 16
- 5.3 가장 긴 팰린드롬 부분 분자열 - Manacher . . . 16
- 5.4 문자열 매칭 - Z . . . 16
- 5.5 접미사 배열 . . . 16

6 계산 기하 **17**
- 6.1 2차원 계산 기하 템플릿 + CCW . . . 17
- 6.2 360도 각도 정렬 . . . 17
- 6.3 다각형 넓이 . . . 17
- 6.4 선분 교차 판정 . . . 17
- 6.5 다각형 내부 판별 . . . 17
- 6.6 볼록 껍질 - Graham Scan . . . 18
- 6.7 가장 먼 두 점 - Rotating Calipers . . . 18
- 6.8 볼록 다각형 내부 판별 . . . 18

7 기타 **18**
- 7.1 가장 긴 증가하는 부분 수열(LIS) . . . 18
- 7.2 이분 탐색 . . . 18
- 7.3 삼분 탐색 . . . 18
- 7.4 C++ 랜덤, GCC 확장, 비트마스킹 트릭 . . . 19
- 7.5 C++ 입력: EOF, 한 줄 입력 / 출력 precision . . . 19
- 7.6 빠른 입력(Fast Input from STDIN) . . . 19
- 7.7 구간별 약수 최대 개수, 최대 소수 . . . 19
- 7.8 카탈란 수, 심슨 적분, 그런디 정리, 픽의 정리, 페르마 포인트, 오일러 정리 . . . 20
- 7.9 경우의 수 - 포함 배제, 스털링 수, 벨 수 . . . 20
- 7.10 삼각형의 오심 - 외심, 내심, 무게중심, 수심, 방심 . . . 20
- 7.11 미적분, 뉴턴 랩슨법 . . . 21
- 7.12 문제 풀이 체크리스트 . . . 21

1 자료 구조

1.1 펜윅 트리(Fenwick Tree / BIT) + 예제

Usage: 1-based 인덱스, 점 갱신과 구간 합
Time Complexity: $O(\log N)$, 공간 $O(N)$

```
// 필요: <vector>, <cassert>, using namespace std;
struct Fenwick {
    int n;
    vector<long long> tree;
    Fenwick(int n) : n(n), tree(n + 1) {}
    void add(int i, long long delta) { // a[i] += delta, 1 <= i <= n
        for (; i <= n; i += i & -i) tree[i] += delta;
    }
    long long prefix(int i) const { // a[1] + ... + a[i], prefix(0) = 0
        long long ret = 0;
        for (; i > 0; i -= i & -i) ret += tree[i];
        return ret;
    }
    long long sum(int l, int r) const { // [l, r], 1 <= l <= r <= n
        return prefix(r) - prefix(l - 1);
    }
};

void fenwick_example() {
    Fenwick ft(5);
    vector<long long> a = {0, 1, 2, 3, 4, 5}; // a[0] 미사용
    for (int i = 1; i <= 5; ++i) ft.add(i, a[i]);
    assert(ft.sum(2, 4) == 9);
    // a[3]을 10으로 대입하려면 기존 값과의 차이를 더한다.
    ft.add(3, 10 - a[3]); a[3] = 10;
    assert(ft.prefix(3) == 13);
    // add(0, ...)는 무한 루프! 인덱스는 반드시 1 이상.
}
```

1.2 bitset 사용법 + 부분집합 합 예제

Usage: 크기는 컴파일 시 결정. 0번 비트는 가장 오른쪽 비트.

```
// 필요: <bitset>, <string>, <cassert>, using namespace std;
void bitset_example() {
    bitset<8> a(string("00101100")), b(5); // b = 00000101
    assert(a[2] && a.test(3)); // 비트 읽기, 유효 인덱스 [0, 8)
    a.set(0); a.reset(2); a.flip(1); // 1로, 0으로, 반전
    a[7] = true;
    auto both = a & b; // 교집합
    auto either = a | b; // 합집합
    auto diff = a ^ b; // 서로 다른 비트
    auto inv = ~a; // 전체 반전
    auto left = a << 2; // 높은 자리로 이동, 넘친 비트 버림
    auto right = a >> 2; // 빈 자리는 0으로 채움
    auto cnt = a.count(); // 1의 개수
    bool any = a.any(), all = a.all(), none = a.none();
    string s = a.to_string(); // 항상 8자리
    auto x = a.to_ullong(); // 범위를 넘으면 overflow_error
    assert(both.to_ulong() == 1 && either.to_ulong() == 175);
    assert(diff.to_ulong() == 174 && inv.to_ulong() == 84);
    assert(left.to_ulong() == 172 && right.to_ulong() == 42);
    assert(cnt == 5 && any && !all && !none);
    assert(s == "10101011" && x == 171);
    a.set(); a.flip(); a.reset(); // 전체 1, 전체 반전, 전체 0

    // 부분집합 합: 각 수를 최대 한 번 사용 (모두 음이 아닌 정수)
    // dp[s] = 합 s를 만들 수 있는가? 0 <= s <= MAX_SUM만 관리
    constexpr int MAX_SUM = 100;
    bitset<MAX_SUM + 1> dp;
    dp[0] = 1;
    for (int v : {3, 5, 8}) dp |= dp << v;
    assert(dp[8] && dp[16] && !dp[7]);
    // 비트 연산은 보통 O(비트 수 / 워드 크기), 공간 O(비트 수).
    // 런타임 크기가 필요하면 vector<uint64_t> 등으로 구현.
}
```

1.3 unordered_set / unordered_map 사용자 정의 해시

Usage: 같은 키는 반드시 같은 해시값을 반환해야 한다.
Time Complexity: 삽입/조회 평균 $O(1)$, 최악 $O(N)$

```
// 필요: <cstdint>, <chrono>, <utility>, <unordered_set>,
//       <unordered_map>, <string>, <cassert>, using namespace std;
struct CustomHash {
    static uint64_t mix(uint64_t x) { // splitmix64
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }
};
struct PairHash {
    size_t operator()(const pair<int, int>& p) const {
        // 두 32비트 정수를 하나의 64비트 값으로 합친다 (음수도 가능).
        uint64_t key = (uint64_t(uint32_t(p.first)) << 32)
                     | uint32_t(p.second);
        return CustomHash{}(key);
    }
};
void custom_hash_example() {
    // set은 두 번째, map은 세 번째 템플릿 인자가 해시 함수 객체.
    unordered_set<long long, CustomHash> st;
    unordered_map<pair<int, int>, string, PairHash> mp;
    st.max_load_factor(0.7f); st.reserve(1000); // 순서에 유의
    mp.max_load_factor(0.7f); mp.reserve(1000);
    st.insert(-42);
    mp[{1, -2}] = "hello";
    assert(st.count(-42) == 1);
```

```
    auto it = mp.find({1, -2}); // 조회만 할 때 []는 새 원소를 만들 수 있음
    assert(it != mp.end() && it->second == "hello");
    // 사용자 struct 키는 operator==도 정의 (또는 KeyEqual 지정).
    // seed는 실행 중 고정. 난수화는 최악 시간 복잡도를 보장하지 않음.
}
```

1.4 세그먼트 트리

Time Complexity: $O(\log N)$

```
// SZ: N보다 크거나 같은 2^k 꼴의 수
// 13만 -> 1 << 17 (131072), 26만 -> 1 << 18 (262144)
// 52만 -> 1 << 19 (524288), 100만 -> 1 << 20 (1048576)
constexpr int SZ = 1 << 20;
ll T[SZ<<1];

void Set(int x, ll v){ // x번째 수를 v로 지정, x는 0 이상 SZ 미만
    x += SZ; T[x] = v;
    while(x /= 2) T[x] = T[x*2] + T[x*2+1];
}

ll Sum(int l, int r){ // [l, r] 구간의 합
    ll res = 0;
    for(l+=SZ, r+=SZ; l<=r; l/=2, r/=2){
        if(l % 2 == 1) res += T[l++];
        if(r % 2 == 0) res += T[r--];
    }
    return res;
}
```

1.5 세그먼트 트리 + 레이지 프로퍼게이션

Time Complexity: $O(\log N)$

```
// SZ: N보다 크거나 같은 2^k 꼴의 수
// 13만 -> 1 << 17 (131072), 26만 -> 1 << 18 (262144)
// 52만 -> 1 << 19 (524288), 100만 -> 1 << 20 (1048576)
constexpr int SZ = 1 << 20;
ll T[SZ<<1], L[SZ<<1];

void Push(int node, int s, int e){
    if(L[node] == 0) return;
    T[node] += (e - s + 1) * L[node];
    if(s != e) L[node*2] += L[node], L[node*2+1] += L[node];
    L[node] = 0;
}

// [l, r]번째 수에 v를 더함, 0 <= l <= r < SZ
void RangeAdd(int l, int r, ll v, int node=1, int s=0, int e=SZ-1){
    Push(node, s, e);
    if(r < s || e < l) return;
    if(l <= s && e <= r){ L[node] += v; Push(node, s, e); return; }
    int m = (s + e) / 2;
    RangeAdd(l, r, v, node*2, s, m);
    RangeAdd(l, r, v, node*2+1, m+1, e);
    T[node] = T[node*2] + T[node*2+1];
}

// [l, r]번째 수의 합을 구함
ll RangeSum(int l, int r, int node=1, int s=0, int e=SZ-1){
    Push(node, s, e);
    if(r < s || e < l) return 0;
    if(l <= s && e <= r) return T[node];
    int m = (s + e) / 2;
    return RangeSum(l, r, node*2, s, m) + RangeSum(l, r, node*2+1, m+1, e);
}
```

1.6 퍼시스턴트 세그먼트 트리

Usage: call init(root[0], s, e) before use

```
struct PSTNode{
  PSTNode *l, *r; int v;
  PSTNode(){ l = r = nullptr; v = 0; }
};
PSTNode *root[101010];
PST(){ memset(root, 0, sizeof root); } // constructor
void init(PSTNode *node, int s, int e){
  if(s == e) return;
  int m = s + e >> 1;
  node->l = new PSTNode; node->r = new PSTNode;
  init(node->l, s, m); init(node->r, m+1, e);
}
void update(PSTNode *prv, PSTNode *now, int s, int e, int x){
  if(s == e){ now->v = prv ? prv->v + 1 : 1; return; }
  int m = s + e >> 1;
  if(x <= m){
    now->l = new PSTNode; now->r = prv->r;
    update(prv->l, now->l, s, m, x);
  }
  else{
    now->r = new PSTNode; now->l = prv->l;
    update(prv->r, now->r, m+1, e, x);
  }
  int t1 = now->l ? now->l->v : 0;
  int t2 = now->r ? now->r->v : 0;
  now->v = t1 + t2;
}
int kth(PSTNode *prv, PSTNode *now, int s, int e, int k){
  if(s == e) return s;
  int m = s + e >> 1, diff = now->l->v - prv->l->v;
  if(k <= diff) return kth(prv->l, now->l, s, m, k);
  else return kth(prv->r, now->r, m+1, e, k-diff);
}
```

2 그래프/트리

2.1 단절점 및 단절선

```
#include <bits/stdc++.h>
```

```cpp
using namespace std;
typedef long long ll;
int n, m, ord[101010], ch[101010], iscut[101010];
vector<int> adj[101010];

int r = 0;
int solve(int v) {
    ord[v] = ++r;
    int fast = 1e9, child = 0;

    for (int i = 0; i < adj[v].size(); i++) {
        int u = adj[v][i];
        if (ch[u] == 1) fast = min(fast, ord[u]);
        else {
            child++;
            ch[u] = 1;
            int prev = solve(u);
            if (v != 1 && prev >= ord[v]) iscut[v] = 1;
            fast = min(fast, prev);
        }
    }

    if (v == 1 && child >= 2) iscut[v] = 1;

    return fast;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    cin >> n >> m;
    for (int i = 1; i <= m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    ch[1] = 1;
    solve(1);

    for (int i = 1; i <= n; i++) cout << iscut[i];

    return 0;
}
```

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int n, m, ord[101010], ch[101010], iscut[202020];
vector< pair<int, int> > adj[101010];

int r = 0;
int solve(int v, int p) {
    ord[v] = ++r;
    int fast = 1e9;

    for (int i = 0; i < adj[v].size(); i++) {
        int u = adj[v][i].first, j = adj[v][i].second;
        if (u == p) continue;
        if (ch[u] == 1) fast = min(fast, ord[u]);
        else {
            ch[u] = 1;
            int prev = solve(u, v);
            if (prev > ord[v]) iscut[j] = 1;
            fast = min(fast, prev);
        }
    }

    return fast;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    cin >> n >> m;
    for (int i = 1; i <= m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back({b, i});
        adj[b].push_back({a, i});
    }

    ch[1] = 1;
    solve(1, 0);

    for (int i = 1; i <= m; i++) cout << iscut[i];

    return 0;
}
```

2.2 SCC - Kosaraju

Time Complexity: $O(V + E)$

```cpp
int N, M, C[10101]; // C[i] = i번 정점이 속한 SCC 번호
vector<int> G[10101], R[10101], V;
vector<vector<int>> S; // 각 SCC에 속한 정점 목록
void AddEdge(int s, int e){
    G[s].push_back(e);
    R[e].push_back(s);
}
void DFS1(int v){
    C[v] = -1;
    for(auto i : G[v]) if(!C[i]) DFS1(i);
    V.push_back(v);
}
void DFS2(int v, int c){
```

```
    C[v] = c; S.back().push_back(v);
    for(auto i : R[v]) if(C[i] == -1) DFS2(i, c);
}
int GetSCC(){ // SCC 개수 반환
    for(int i=1; i<=N; i++) if(!C[i]) DFS1(i);
    reverse(V.begin(), V.end());
    int cnt = 0;
    for(auto i : V) if(C[i] == -1) S.emplace_back(), DFS2(i, cnt++);
    return cnt;
} // 각 SCC는 위상 정렬 순서대로 번호 매겨져 있음
```

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int n, m;
vector<int> adj1[202020], adj2[202020];

int ch[202020];
stack<int> stk;
vector< vector<int> > scc;
void dfs1(int v) {
    ch[v] = 1;
    for (int i = 0; i < adj1[v].size(); i++) {
        int u = adj1[v][i];
        if (ch[u]) continue;
        dfs1(u);
    }
    stk.push(v);
}

void dfs2(int v) {
    ch[v] = 1;
    scc.back().push_back(v);
    for (int i = 0; i < adj2[v].size(); i++) {
        int u = adj2[v][i];
        if (ch[u]) continue;
        dfs2(u);
    }
}

bool res;
set<int> S;
void check(int v) {
    ch[v] = 1;
    if (S.find(v%n) == S.end()) S.insert(v%n);
    else res = false;
    for (int i = 0; i < adj1[v].size(); i++) {
        int u = adj1[v][i];
        if (ch[u]) continue;
        check(u);
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    while (scanf("%d %d", &n, &m) == 2) {
        bool ans = true;
        scc.clear();
        while (stk.size() > 0) stk.pop();

        for (int i = 1; i <= 2*n; i++) adj1[i].clear();
        for (int i = 1; i <= 2*n; i++) adj2[i].clear();

        for (int i = 0; i < m; i++) {
            int a, b, na, nb;
            scanf("%d %d", &a, &b);
            na = -a; nb = -b;
            if (a < 0) a = -a+n;
            if (b < 0) b = -b+n;
            if (na < 0) na = -na+n;
            if (nb < 0) nb = -nb+n;
            adj1[na].push_back(b);
            adj1[nb].push_back(a);
            adj2[b].push_back(na);
            adj2[a].push_back(nb);
        }

        for (int i = 1; i <= 2*n; i++) ch[i] = 0;
        for (int i = 1; i <= 2*n; i++) {
            if (ch[i]) continue;
            dfs1(i);
        }

        for (int i = 1; i <= 2*n; i++) ch[i] = 0;
        while (stk.size() > 0) {
            int v = stk.top();
            stk.pop();
            if (ch[v]) continue;
            scc.push_back(vector<int>());
            dfs2(v);
        }

        for (int i = 0; i < scc.size(); i++) {
            set<int> s;
            for (int j = 0; j < scc[i].size(); j++) {
                if (s.find(scc[i][j]%n) == s.end()) s.insert(scc[i][j]%n);
                else ans = false;
            }
        }

        for (int i = 1; i <= 2*n; i++) ch[i] = 0;
        res = true;
        S.clear();
        check(1);
        if (!res) ans = false;

        cout << (ans ? "yes\n" : "no\n");
```

```
    }

    return 0;
}
```

2.3 BCC - Tarjan

Time Complexity: $O(V+E)$

```
// 1-based, 다른 거 호출하기 전에 tarjan 먼저 호출해야 함
vector<int> G[MAX_V]; int In[MAX_V], Low[MAX_V], P[MAX_V];
void addEdge(int s, int e){ G[s].push_back(e); G[e].push_back(s); }
void tarjan(int n){ /// Pre-Process
  int pv = 0;
  function<void(int,int)> dfs = [&pv,&dfs](int v, int b){
    In[v] = Low[v] = ++pv; P[v] = b;
    for(auto i : G[v]){
      if(i == b) continue;
      if(!In[i]) dfs(i, v), Low[v] = min(Low[v], Low[i]); else Low[v] = min(Low[v], In[i]);
    }
  };
  for(int i=1; i<=n; i++) if(!In[i]) dfs(i, -1);
}
vector<int> cutVertex(int n){
  vector<int> res; array<char,MAX_V> isCut; isCut.fill(0);
  function<void(int)> dfs = [&dfs,&isCut](int v){
    int ch = 0;
    for(auto i : G[v]){
      if(P[i] != v) continue; dfs(i); ch++;
      if(P[v] == -1 && ch > 1) isCut[v] = 1; else if(P[v] != -1 && Low[i] >= In[v])
      isCut[v]=1;
    }
  };
  for(int i=1; i<=n; i++) if(P[i] == -1) dfs(i);
  for(int i=1; i<=n; i++) if(isCut[i]) res.push_back(i);
  return move(res);
}
vector<PII> cutEdge(int n){
  vector<PII> res;
  function<void(int)> dfs = [&dfs,&res](int v){
    for(int t=0; t<G[v].size(); t++){
      int i = G[v][t]; if(t != 0 && G[v][t-1] == G[v][t]) continue;
      if(P[i] != v) continue; dfs(i);
      if((t+1 == G[v].size() || i != G[v][t+1]) && Low[i] > In[v])
      res.emplace_back(min(v,i), max(v,i));
    }
  };
  for(int i=1; i<=n; i++) sort(G[i].begin(), G[i].end()); // multi edge -> sort
  for(int i=1; i<=n; i++) if(P[i] == -1) dfs(i);
  return move(res); // sort(all(res));
}
vector<int> BCC[MAX_V]; // BCC[v] = components which contains v
void vertexDisjointBCC(int n){ // allow multi edge, not allow self loop
  int cnt = 0; array<char,MAX_V> vis; vis.fill(0);
  function<void(int,int)> dfs = [&dfs,&vis,&cnt](int v, int c){
    vis[v] = 1; if(c > 0) BCC[v].push_back(c);
    for(auto i : G[v]){
      if(vis[i]) continue;
      if(In[v] <= Low[i]) BCC[v].push_back(++cnt), dfs(i, cnt); else dfs(i, c);
    }
  };
  for(int i=1; i<=n; i++) if(!vis[i]) dfs(i, 0);
  for(int i=1; i<=n; i++) if(BCC[i].empty()) BCC[i].push_back(++cnt);
}
```

2.4 최대 유량 - Dinic

Time Complexity: $O(V^2E)$, 모든 간선의 용량이 1이면 $O(\min(V^{2/3}, E^{1/2})E)$

```
// Directed Graph이면 add_edge(s, e, c)
// Undirected Graph이면 add_edge(s, e, c, c)
template<typename flow_t, flow_t MAX_U=(1<<30)>
struct Dinic{ // 0-based
  struct edge_t{ int v, r; flow_t c, f; };
  int n;
  vector<vector<edge_t>> g;
  vector<int> lv, idx;
  Dinic(int n) : n(n) { clear(); }
  void clear(){
    g = vector<vector<edge_t>>(n);
    lv = vector<int>(n, 0);
    idx = vector<int>(n, 0);
  }
  void add_edge(int s, int e, flow_t c1, flow_t c2=flow_t(0)){
    g[s].push_back({e, (int)g[e].size(), c1, 0});
    g[e].push_back({s, (int)g[s].size()-1, c2, 0});
  }
  bool bfs(int s, int t, flow_t limit=1){
    fill(lv.begin(), lv.end(), 0);
    queue<int> que; que.push(s); lv[s] = 1;
    while(!que.empty()){
      int v = que.front(); que.pop();
      for(const auto &e : g[v]) if(!lv[e.v] && e.c - e.f >= limit) que.push(e.v), lv[e.v] =
      lv[v] + 1;
    }
    return lv[t] != 0;
  }
  flow_t dfs(int v, int t, flow_t fl=MAX_U){
    if(v == t || fl == flow_t(0)) return fl;
    for(int &i=idx[v]; i<g[v].size(); i++){
      auto &e = g[v][i];
      if(lv[e.v] != lv[v] + 1 || e.c - e.f == flow_t(0)) continue;
      flow_t now = dfs(e.v, t, min(fl, e.c - e.f));
      if(now == flow_t(0)) continue;
      e.f += now; g[e.v][e.r].f -= now;
      return now;
    }
    return 0;
  }
  flow_t maximum_flow(int s, int t){
```

```
    flow_t flow = 0, augment = 0;
    while(bfs(s, t)){
      fill(idx.begin(), idx.end(), 0);
      while((augment=dfs(s, t)) != flow_t(0)) flow += augment;
    }
    return flow;
  }
  // {최소 컷 비용, s와 같은 집합, t와 같은 집합, 절단 간선}
  tuple<flow_t, vector<int>, vector<int>, vector<pair<int,int>>> minimum_cut(int s, int t){
    flow_t flow = maximum_flow(s, t);
    vector<int> a, b;
    vector<pair<int,int>> edges;
    bfs(s, t, 1);
    for(int i=0; i<n; i++) (lv[i] ? a : b).push_back(i);
    for(auto i : a) for(auto e : g[i]) if(e.c != flow_t(0) && !lv[e.v])
    edges.emplace_back(i, e.v);
    return {flow, a, b, edges};
  }
};
```

2.5 MCMF

```
// 유량을 k 만큼 흘리는 경우: run(src, snk, k)
// 유량을 최대한 많이 흘리는 경우: run(src, snk)
template<typename flow_t=int, typename cost_t=long long, flow_t MAX_U=(1<<30), cost_t
MAX_C=(1LL<<60)>
struct MinCostFlow{ // 0-based
    struct edge_t{ int v, r; flow_t c; cost_t d; };
    int n;
    vector<vector<edge_t>> g;
    vector<int> prv, idx, chk;
    vector<cost_t> dst;
    MinCostFlow(int n) : n(n) { clear(); }
    void clear(){
        g = vector<vector<edge_t>>(n);
        prv = idx = chk = vector<int>(n);
        dst = vector<cost_t>(n);
    }
    void add_edge(int s, int e, flow_t c, cost_t d){
        g[s].push_back({e, (int)g[e].size(), c, d});
        g[e].push_back({s, (int)g[s].size()-1, 0, -d});
    }
    bool find_path(int s, int t){
        fill(chk.begin(), chk.end(), 0);
        fill(dst.begin(), dst.end(), MAX_C);
        queue<int> que; que.push(s); dst[s] = 0; chk[s] = 1;
        while(!que.empty()){
            int v = que.front(); que.pop(); chk[v] = 0;
            for(int i=0; i<g[v].size(); i++){
                const auto &e = g[v][i];
                if(e.c > 0 && dst[e.v] > dst[v] + e.d){
                    dst[e.v] = dst[v] + e.d; prv[e.v] = v; idx[e.v] = i;
                    if(!chk[e.v]) que.push(e.v), chk[e.v] = 1;
                }
            }
        }
        return dst[t] < MAX_C;
    }
    pair<flow_t, cost_t> augment(int s, int t, flow_t k=-1){
        if(!find_path(s, t)) return {0, 0};
        flow_t fl = MAX_U;
        for(int i=t; i!=s; i=prv[i]) fl = min(fl, g[prv[i]][idx[i]].c);
        if(k != -1) fl = min(fl, k);
        for(int i=t; i!=s; i=prv[i]){
            g[prv[i]][idx[i]].c -= fl;
            g[i][g[prv[i]][idx[i]].r].c += fl;
        }
        return {fl, fl * dst[t]};
    }
    pair<flow_t, cost_t> run(int s, int t, flow_t k=-1){
        flow_t flow = 0; cost_t cost = 0;
        while(true){
            auto [fl,cst] = augment(s, t, k);
            if(fl == 0) break;
            flow += fl; cost += cst;
            if(k != -1) k -= fl;
        }
        return {flow, cost};
    }
};
```

2.6 이분 매칭 - Hopcroft Karp

Time Complexity: $O(E\sqrt{V})$

```
// n: 왼쪽 정점 개수, m: 오른쪽 정점 개수, 0-based
struct HopcroftKarp{
  int n, m;
  vector<vector<int>> g;
  vector<int> dst, le, ri;
  vector<char> visit, track;
  HopcroftKarp(int n, int m) : n(n), m(m) { clear(); }
  void clear(){
    g = vector<vector<int>>(n); dst = vector<int>(n, 0);
    le = vector<int>(n, -1); ri = vector<int>(m, -1);
    visit = vector<char>(n, 0); track = vector<char>(n+m, 0);
  }
  void add_edge(int s, int e){ g[s].push_back(e); }
  bool bfs(){
    bool res = false;
    queue<int> que;
    fill(dst.begin(), dst.end(), 0);
    for(int i=0; i<n; i++) if(le[i] == -1) que.push(i), dst[i] = 1;
    while(!que.empty()){
      int v = que.front(); que.pop();
      for(auto i : g[v]){
        if(ri[i] == -1) res = true;
        else if(!dst[ri[i]]) dst[ri[i]] = dst[v] + 1, que.push(ri[i]);
      }
    }
```

```
    return res;
  }
  bool dfs(int v){
    if(visit[v]) return false;
    visit[v] = 1;
    for(auto i : g[v]){
      if(ri[i] == -1 || !visit[ri[i]] && dst[ri[i]] == dst[v] + 1 && dfs(ri[i])){
        le[v] = i; ri[i] = v; return true;
      }
    }
    return false;
  }
  int maximum_matching(){
    int res = 0;
    fill(le.begin(), le.end(), -1);
    fill(ri.begin(), ri.end(), -1);
    while(bfs()){
      fill(visit.begin(), visit.end(), 0);
      for(int i=0; i<n; i++) if(le[i] == -1) res += dfs(i);
    }
    return res;
  }
  vector<pair<int,int>> maximum_matching_edges(){
    int matching = maximum_matching();
    vector<pair<int,int>> edges; edges.reserve(matching);
    for(int i=0; i<n; i++) if(le[i] != -1) edges.emplace_back(i, le[i]);
    return edges;
  }
  void dfs_track(int v){
    if(track[v]) return; track[v] = 1;
    for(auto i : g[v]) track[n+i] = 1, dfs_track(ri[i]);
  }
  tuple<vector<int>, vector<int>, int> minimum_vertex_cover(){
    int matching = maximum_matching();
    fill(track.begin(), track.end(), 0);
    for(int i=0; i<n; i++) if(le[i] == -1) dfs_track(i);
    vector<int> lv, rv;
    for(int i=0; i<n; i++) if(!track[i]) lv.push_back(i);
    for(int i=0; i<m; i++) if(track[n+i]) rv.push_back(i);
    assert(lv.size() + rv.size() == matching);
    return {lv, rv, lv.size() + rv.size()};
  }
};
```

2.7 최소 공통 조상(LCA)

Time Complexity: 전처리 $O(N \log N)$, 쿼리 $O(\log N)$

```
int N, Q, D[101010], P[22][101010];
vector<int> G[101010];
void Connect(int u, int v){
    G[u].push_back(v); G[v].push_back(u);
}
void DFS(int v, int b=-1){
    for(auto i : G[v]) if(i != b) D[i] = D[v] + 1, P[0][i] = v, DFS(i, v);
}
int LCA(int u, int v){
    if(D[u] < D[v]) swap(u, v);
    int diff = D[u] - D[v];
    for(int i=0; diff; i++, diff>>=1) if(diff & 1) u = P[i][u];
    if(u == v) return u;
    for(int i=21; i>=0; i--) if(P[i][u] != P[i][v]) u = P[i][u], v = P[i][v];
    return P[0][u];
}
////
// 1. Connect로 간선 추가
// 2. DFS(1) 호출
// 3. 아래 코드 실행
for(int i=1; i<22; i++) for(int j=1; j<=N; j++) P[i][j] = P[i-1][P[i-1][j]];
// 4. LCA(u, v)로 최소 공통 조상 구할 수 있음
```

2.8 Heavy Light Decomposition

Time Complexity: 전처리 $O(N)$, 쿼리 $O(T(N) \log N)$

```
int N, Q, A[SZ], Top[SZ], Par[SZ], Dep[SZ], Sz[SZ], In[SZ];
vector<int> Inp[SZ], G[SZ];

void Connect(int u, int v){
    Inp[u].push_back(v); Inp[v].push_back(u);
}

void DFS0(int v, int b=-1){
    for(auto i : Inp[v]) if(i != b)
        Dep[i] = Dep[v] + 1, Par[i] = v, G[v].push_back(i), DFS0(i, v);
}

void DFS1(int v){
    Sz[v] = 1;
    for(auto &i : G[v]){
        DFS1(i); Sz[v] += Sz[i];
        if(Sz[i] > Sz[G[v][0]]) swap(i, G[v][0]);
    }
}

void DFS2(int v){
    static int pv = 0; In[v] = ++pv;
    for(auto i : G[v]) Top[i] = i == G[v][0] ? Top[v] : i, DFS2(i);
}

void VertexUpdate(int x, int v){
    Update(In[x], v);
}

long long PathQuery(int u, int v){
    long long res = 0;
    for(; Top[u] != Top[v]; u=Par[Top[u]]){
        if(Dep[Top[u]] < Dep[Top[v]]) swap(u, v);
        res += Query(In[Top[u]], In[u]);
    }
```

```
    if(In[u] > In[v]) swap(u, v);
    res += SegQuery(In[u], In[v]); // 정점 쿼리는 In[u], 간선 쿼리는 In[u]+1
    return res;
}
/////
// 1. Connect로 간선 추가
// 2. DFS0(1); DFS1(1); DFS2(Top[1]=1); 호출
// 3. VertexUpdate, PathQuery로 연산 수행
//  3-1. Update, Query는 배열의 구간 쿼리를 지원하는 자료구조(ex. 세그먼트 트리) 사용
```

2.9 센트로이드

```
#include <bits/stdc++.h>
using namespace std;
int n, sz[200010];
long long int a[200010], b[200010], ans, chk[200010];
vector<int> v[200010];
map<long long int, int> m;
int get_size(int x, int y)
{
    sz[x]=1;
    for(int e:v[x]){
        if(e!=y&&chk[e]==0) sz[x]+=get_size(e, x);
    }
    return sz[x];
}
int get_centroid(int x, int y, int z)
{
    for(int e:v[x]){
        if(e!=y&&chk[e]==0&&z<sz[e]) return get_centroid(e, x, z);
    }
    return x;
}
void cal(int x, int y, int z)
{
    ans+=m[-z];
    for(int e:v[x]){
        if(e!=y&&chk[e]==0) cal(e, x, z+a[e]);
    }
}
void update(int x, int y, int z)
{
    m[z]+=1;
    for(int e:v[x]){
        if(e!=y&&chk[e]==0){
            update(e, x, z+a[e]);
        }
    }
}
void cd(int x, int y)
{
    int z=get_size(x, y);
    int c=get_centroid(x, y, z/2);
    chk[c]=1;
    m.clear();
    for(int e:v[c]){
        if(chk[e]==0){
            cal(e, c, a[c]+a[e]);
            update(e, c, a[e]);
        }
    }
    ans+=m[-a[c]];
    for(int e:v[c]){
        if(chk[e]==0) cd(e, c);
    }
}
int main()
{
    ios_base::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    cin>>n;
    for(int i=1; i<=n; i++){
        char c;
        cin>>c;
        if(c=='U') a[i]=200001;
        if(c=='C') a[i]=-200002;
        if(c=='P') a[i]=200003;
    }
    for(int i=1; i<n; i++){
        int x, y;
        cin>>x>>y;
        v[x].push_back(y);
        v[y].push_back(x);
    }
    cd(1, 0);
    cout<<ans;
}
```

3 동적계획법

3.1 Convex Hull Trick

Usage: call init() before use

```
// 직선 개수 N, 쿼리 횟수 Q일 때 O(N+Q)
// (최댓값 쿼리) 삽입하는 직선의 기울기는 단조 증가해야 함
// (최솟값 쿼리) 삽입하는 직선의 기울기는 단조 감소해야 함
// 쿼리를 하는 x좌표는 단조 증가해야 함
struct Line{
  ll a, b, c; // y = ax + b, c = line index
  Line(ll a, ll b, ll c) : a(a), b(b), c(c) {}
  ll f(ll x){ return a * x + b; }
};
vector<Line> v; int pv;
void init(){ v.clear(); pv = 0; }
int chk(const Line &a, const Line &b, const Line &c) const {
  return (__int128_t)(a.b - b.b) * (b.a - c.a) <= (__int128_t)(c.b - b.b) * (b.a - a.a);
}
void insert(Line l){
  if(v.size() > pv && v.back().a == l.a){
```

```
    // if min query, then if(l.b > v.back().b)
    if(l.b < v.back().b) l = v.back();
    v.pop_back();
  }
  while(v.size() >= pv+2 && chk(v[v.size()-2], v.back(), l)) v.pop_back();
  v.push_back(l);
}
p query(ll x){ // if min query, then v[pv].f(x) >= v[pv+1].f(x)
  while(pv+1 < v.size() && v[pv].f(x) <= v[pv+1].f(x)) pv++;
  return {v[pv].f(x), v[pv].c};
}
///// line container start (max query) /////
// 직선 개수 N 쿼리 개수 Q일 때 O((N+Q) log N)
// 단조성 제약 조건 없음
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
}; // (for doubles, use inf = 1/.0, div(a,b) = a/b)
struct LineContainer : multiset<Line, less<>> {
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b); } // floor
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
  }
  ll query(ll x) { assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

3.2 aliens trick + monotone queue optimization

```
#include <bits/stdc++.h>
using namespace std;
#define int long long int
int n, k;
int a[50010], dp[50010], dis[50010];
int cost(int x, int y)
{
    return dp[x]+(y-x)*(a[y]-a[x]);
}
int cross(int x, int y)
{
    int l=y+1, r=n+1;
    while(1){
        if(l>=r) return l-1;
        int mid=(l+r)/2;
        if(cost(x, mid)<cost(y, mid)) l=mid+1;
        else r=mid;
    }
}
void monotone(int m)
{
    deque<int> dq;
    for(int i=1; i<=n; i++){
        while(dq.size()>=2){
            int x=dq[dq.size()-2];
            int y=dq[dq.size()-1];
            if(cross(x, y)>=cross(y, i-1)) dq.pop_back();
            else break;
        }
        dq.push_back(i-1);
        while(dq.size()>=2){
            int x=dq[0], y=dq[1];
            if(cross(x, y)<i) dq.pop_front();
            else break;
        }
        dis[i]=dis[dq[0]]+1;
        dp[i]=cost(dq[0], i)+m;
    }
}
signed main()
{
    ios_base::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    cin>>n>>k;
    for(int i=1; i<=n; i++) cin>>a[i], a[i]+=a[i-1];
    if(k>=n){
        cout<<a[n];
        return 0;
    }
    int l=0, r=2e14;
    while(l<r){
        int mid=(l+r)/2;
        monotone(mid);
        if(dis[n]>k) l=mid+1;
        else r=mid;
    }
    monotone(l);
    cout<<dp[n]-k*l;
}
void monotone(int m)
{
    int x=0, y=1;
    q[0]=0, c[0]=n;
    for(int i=1; i<=n; i++){
        while(c[x]<i) x++;
        dp[i]=cost(q[x], i)+m;
        dis[i]=dis[q[x]]+1;
        while(x+1<y&&c[y-2]>=cross(q[y-1], i)) y--;
```

```
        c[y-1]=cross(q[y-1], i), q[y]=i, c[y++]=n;
    }
}
```

3.3 Dnc optimization

```
#include <bits/stdc++.h>
using namespace std;
int l, g;
long long c[8010], dp[810][8010];
void f(int i, int l, int r, int s, int e)
{
    if(l>r) return;
    int mid=(l+r)/2, opt=-1;
    dp[i][mid]=1e18;
    for(int k=s; k<=min(e, mid-1); k++){
        long long x=dp[i-1][k]+(c[mid]-c[k])*(mid-k);
        if(dp[i][mid]>x){
            dp[i][mid]=x, opt=k;
        }
    }
    f(i, l, mid-1, s, opt);
    f(i, mid+1, r, opt, e);
}
int main()
{
    ios_base::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    cin>>l>>g;
    g=min(g, l);
    for(int i=1; i<=l; i++) cin>>c[i], c[i]+=c[i-1];
    for(int i=1; i<=l; i++) dp[1][i]=i*c[i];
    for(int i=2; i<=g; i++) f(i, i, l, i-1, l-1);
    cout<<dp[g][l];
}
```

4 수학

4.1 나눗셈, 최대공약수, 최소공배수

```
// floor(p / q)
int floor(int p, int q){
    if(q < 0) p = -p, q = -q;
    return p >= 0 ? p / q : (p - q + 1) / q;
}

// ceil(p / q)
int ceil(int p, int q){
    if(q < 0) p = -p, q = -q;
    return p >= 0 ? (p + q - 1) / q : p / q;
}

// a, b >= 0, O(log max(a,b))
// or use std::gcd, std::lcm in C++17
int gcd(int a, int b){ return b ? gcd(b, a % b) : a; }
int lcm(int a, int b){ return a / gcd(a, b) * b; }
```

4.2 빠른 거듭제곱

Usage: $a^b \pmod c$를 구하는 함수
Time Complexity: $O(\log b)$

```
ll PowMod(ll a, ll b, ll c){
    if(c == 1) return 0;
    ll res = 1;
    for(a%=c; b; b >>= 1, a = a * a % c) if(b & 1) res = res * a % c;
    return res;
}
```

4.3 소수 판별, 소인수분해

Time Complexity: $O(\sqrt{N})$

```
// IsPrime(2) = true, IsPrime(4) = false
// Factorize(72) = { {2, 3}, {3, 2} }
bool IsPrime(ll n){
    if(n < 2) return false;
    for(ll i=2; i*i<=n; i++) if(n % i == 0) return false;
    return true;
}
vector<pair<ll,ll>> Factorize(ll n){
    if(n == 1) return {};
    vector<pair<ll,ll>> res;
    for(ll i=2; i*i<=n; i++){
        if(n % i != 0) continue;
        int cnt = 0;
        while(n % i == 0) n /= i, cnt++;
        res.emplace_back(i, cnt);
    }
    if(n != 1) res.emplace_back(n, 1);
    return res;
}
```

4.4 에라토스테네스의 체, 소인수분해

Time Complexity: Sieve: $O(N \log\log N)$, Factorize: $O(\log N)$

```
int SP[5050505]; // SP[i] = i의 가장 작은 소인수
vector<int> Primes;

// n 이하의 모든 소수를 구함
void Sieve(int n){
  for(int i=2; i<=n; i++){
    if(SP[i]) continue;
    for(int j=i; j<=n; j+=i) if(!SP[j]) SP[j] = i;
  }
}

// Sieve 먼저 호출해야 함
vector<pair<int,int>> Factorize(int n){
  vector<pair<int,int>> res;
```

```
  while(n != 1){
    if(res.empty() || res.back().first != SP[n]) res.emplace_back(SP[n], 1);
    else res.back().second++;
    n /= SP[n];
  }
  return res;
}
```

4.5 선형 시간 체, 곱셈적 함수 전처리

```
// sigma 계산하는 부분 제외하면 O(n), sigma 계산은 O(n log log n)
// pw(j, e[i*j]) 모두 전처리하면 O(n)에 계산할 수 있음
// prime: n 이하 소수 목록
// sp : 최소 소인수, 소수라면 0
// tau : 약수 개수, sigma : 약수 합
// phi : n 이하 자연수 중 n과 서로소인 개수
// mu : 2번 이상 곱해진 소인수가 있으면 0, 그렇지 않으면 (-1)^(소인수 개수)
// e[i] : i에 sp[i]가 곱해진 횟수
vector<int> prime;
int sp[sz], e[sz], phi[sz], mu[sz], tau[sz], sigma[sz];
phi[1] = mu[1] = tau[1] = sigma[1] = 1;
for(int i=2; i<=n; i++){
  if(!sp[i]){
    prime.push_back(i);
    e[i] = 1; phi[i] = i-1; mu[i] = -1; tau[i] = 2; sigma[i] = i+1;
  }
  for(auto j : prime){
    if(i*j >= sz) break;
    sp[i*j] = j;
    if(i % j == 0){
      e[i*j] = e[i]+1; phi[i*j] = phi[i]*j; mu[i*j] = 0;
      tau[i*j] = tau[i]/e[i*j]*(e[i*j]+1);
      sigma[i*j] = sigma[i]*(j-1)/(pw(j, e[i*j])-1)*(pw(j, e[i*j]+1)-1)/(j-1);//overflow
      break;
    }
    e[i*j] = 1; phi[i*j] = phi[i] * phi[j]; mu[i*j] = mu[i] * mu[j];
    tau[i*j] = tau[i] * tau[j]; sigma[i*j] = sigma[i] * sigma[j];
  }
}
```

4.6 확장 유클리드 알고리즘

Time Complexity: $O(\log \max(a, b))$

```
// 정수 a, b 주어지면 ax + by = gcd(a, b) = g
// 를 만족하는 정수 {g, x, y} 반환
tuple<ll,ll,ll> ext_gcd(ll a, ll b){
  if(b == 0) return {a, 1, 0};
  auto [g,x,y] = ext_gcd(b, a % b);
  return {g, y, x - a/b * y};
}
```

4.7 중국인의 나머지 정리

Time Complexity: $O(k \log m)$

```
// a = a1 (mod m1), a = a2 (mod m2)를 만족하는 {a, lcm(m1, m2)} 반환
// 만약 a가 존재하면 0 <= a < lcm(m1, m2) 에서 유일하게 존재
// a가 존재하지 않는 경우 {-1, -1} 반환
ll mod(ll a, ll b){ return (a %= b) >= 0 ? a : a + b; }
pair<ll,ll> crt(ll a1, ll m1, ll a2, ll m2){
  ll g = gcd(m1, m2), m = m1 / g * m2;
  if((a2 - a1) % g) return {-1, -1};
  ll md = m2/g, s = mod((a2-a1)/g, m2/g);
  ll t = mod(get<1>(ext_gcd(m1/g%md, m2/g)), md);
  return { a1 + s * t % md * m1, m };
}
// a = a_i (mod m_i)를 만족하는 {a, lcm(m_1, ... , m_k)} 반환
// a가 존재하지 않는 경우 {-1, -1} 반환
pair<ll,ll> crt(const vector<ll> &a, const vector<ll> &m){
  ll ra = a[0], rm = m[0];
  for(int i=1; i<m.size(); i++){
    auto [aa,mm] = crt(ra, rm, a[i], m[i]);
    if(mm == -1) return {-1, -1}; else tie(ra,rm) = tie(aa,mm);
  }
  return {ra, rm};
}
```

4.8 이항 계수를 소수로 나눈 나머지

Time Complexity: 전처리 $O(P)$, 쿼리 $O(\log P)$

```
// Lucas C(13);
// C.calc(5, 3) = 5C3 = 10
// C.calc(10, 2) = 10C2 % 13 = 45 % 13 = 6
// 주의: P는 소수
// P가 크고(약 10억) n,r이 작으면(1000만 이하)
// 생성자에서 fac, inv를 1000만까지만 구해도 됨

struct Lucas{ // init : O(P), query : O(log P)
    const size_t P;
    vector<ll> fac, inv;
    ll Pow(ll a, ll b){
        ll res = 1;
        for(; b; b>>=1, a=a*a%P) if(b & 1) res = res * a % P;
        return res;
    }
    Lucas(size_t P) : P(P), fac(P), inv(P) {
        fac[0] = 1; for(int i=1; i<P; i++) fac[i] = fac[i-1] * i % P;
        inv[P-1] = Pow(fac[P-1], P-2); for(int i=P-2; ~i; i--) inv[i] = inv[i+1] * (i+1) %
        P;
    }
    ll small(ll n, ll r) const { return r <= n ? fac[n] * inv[r] % P * inv[n-r] % P : 0LL;
    }
    ll calc(ll n, ll r) const {
        if(n < r || n < 0 || r < 0) return 0;
        if(!n || !r || n == r) return 1; else return small(n%P, r%P) * calc(n/P, r/P) % P;
    }
};
```

4.9 빠른 소수 판별, 소인수분해 - Miller Rabin, Pollard Rho

Usage: 처음에 Sieve() 호출해야 함
Time Complexity: IsPrime: $O(\log^2 N)$, Factorize: 약 $O(N^{1/4})$

```
constexpr int SZ = 10'000'000;
bool PrimeCheck[SZ+1]; vector<int> Primes;
void Sieve(){
    memset(PrimeCheck, true, sizeof PrimeCheck);
    PrimeCheck[0] = PrimeCheck[1] = false;
    for(int i=2; i<=SZ; i++){
        if(PrimeCheck[i]) Primes.push_back(i);
        for(auto j : Primes){
            if(i*j > SZ) break;
            PrimeCheck[i*j] = false;
            if(i % j == 0) break;
        }
    }
}

using ll = long long;
using ull = unsigned long long;
ull MulMod(ull a, ull b, ull c){ return (__uint128_t)a * b % c; }
ull PowMod(ull a, ull b, ull c){
    ull res = 1; a %= c;
    for(; b; b>>=1, a=MulMod(a,a,c)) if(b & 1) res = MulMod(res,a,c);
    return res;
}
bool MillerRabin(ull n, ull a){
    if(a % n == 0) return true;
    int cnt = __builtin_ctzll(n - 1);
    ull p = PowMod(a, n >> cnt, n);
    if(p == 1 || p == n - 1) return true;
    while(cnt--) if((p=MulMod(p,p,n)) == n - 1) return true;
    return false;
}
bool IsPrime(ll n){
    if(n <= SZ) return PrimeCheck[n];
    if(n <= 2) return n == 2;
    if(n % 2 == 0 || n % 3 == 0 || n % 5 == 0 || n % 7 == 0 || n % 11 == 0) return false;
    // 32bit integer: {2, 7, 61}
    for(int p : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) if(!MillerRabin(n, p))
    return false;
    return true;
}
ll Rho(ll n){
    while(true){
        ll x = rand() % (n - 2) + 2, y = x, c = rand() % (n - 1) + 1;
        while(true){
            x = (MulMod(x, x, n) + c) % n;
            y = (MulMod(y, y, n) + c) % n;
            y = (MulMod(y, y, n) + c) % n;
            ll d = __gcd(abs(x - y), n);
            if(d == 1) continue;
            if(IsPrime(d)) return d;
            else{ n = d; break; }
        }
    }
}
vector<pair<ll,ll>> Factorize(ll n){
    vector<pair<ll,ll>> v;
    int two = __builtin_ctzll(n);
    if(two > 0) v.emplace_back(2, two), n >>= two;
    if(n == 1) return v;
    while(!IsPrime(n)){
        ll d = Rho(n), cnt = 0;
        while(n % d == 0) cnt++, n /= d;
        v.emplace_back(d, cnt);
        if(n == 1) break;
    }
    if(n != 1) v.emplace_back(n, 1);
    return v;
}
```

4.10 가우스 소거법 - RREF, 랭크, 행렬식, 역행렬

Time Complexity: $O(N^3)$

```
// T Add(T a, T b), Sub, Mul, Div 구현해야 함
// bool IsZero(T x) 구현해야 함
template<typename T> // return {rref, rank, det, inv}
tuple<vector<vector<T>>, int, T, vector<vector<T>>> Gauss(vector<vector<T>> a, bool
square=true){
  int n = a.size(), m = a[0].size(), rank = 0;
  vector<vector<T>> out(n, vector<T>(m, 0)); T det = T(1);
  for(int i=0; i<n; i++) if(square) out[i][i] = T(1);
  for(int i=0; i<m; i++){
    if(rank == n) break;
    if(IsZero(a[rank][i])){
      T mx = T(0); int idx = -1; // fucking precision error
      for(int j=rank+1; j<n; j++) if(mx < abs(a[j][i])) mx = abs(a[j][i]), idx = j;
      if(idx == -1 || IsZero(a[idx][i])){ det = 0; continue; }
      for(int k=0; k<m; k++){
        a[rank][k] = Add(a[rank][k], a[idx][k]);
        if(square) out[rank][k] = Add(out[rank][k], out[idx][k]);
      }
    }
    det = Mul(det, a[rank][i]);
    T coeff = Div(T(1), a[rank][i]);
    for(int j=0; j<m; j++) a[rank][j] = Mul(a[rank][j], coeff);
    for(int j=0; j<m; j++) if(square) out[rank][j] = Mul(out[rank][j], coeff);
    for(int j=0; j<n; j++){
      if(rank == j) continue;
      T t = a[j][i]; // Warning: [j][k], [rank][k]
      for(int k=0; k<m; k++) a[j][k] = Sub(a[j][k], Mul(a[rank][k], t));
      for(int k=0; k<m; k++) if(square) out[j][k] = Sub(out[j][k], Mul(out[rank][k], t));
    }
    rank++;
  }
  return {a, rank, det, out};
}
```

4.11 다항식 곱셈(FFT)

Time Complexity: $O(N \log N)$

```
// 104,857,601   =  25 * 2^22 + 1, w = 3 | 998,244,353   = 119 * 2^23 + 1, w = 3
// 2,281,701,377 =  17 * 2^27 + 1, w = 3 | 2,483,027,969 =  37 * 2^26 + 1, w = 3
// 2,113,929,217 =  63 * 2^25 + 1, w = 5 | 1,092,616,193 = 521 * 2^21 + 1, w = 3
using real_t = double; using cpx = complex<real_t>;
void FFT(vector<cpx> &a, bool inv_fft=false){
  int N = a.size(); vector<cpx> root(N/2);
  for(int i=1, j=0; i<N; i++){
    int bit = N / 2;
    while(j >= bit) j -= bit, bit >>= 1;
    if(i < (j += bit)) swap(a[i], a[j]);
  }
  long double ang = 2 * acosl(-1) / N * (inv_fft ? -1 : 1);
  for(int i=0; i<N/2; i++) root[i] = cpx(cosl(ang*i), sinl(ang*i));
  /*
  NTT : ang = pow(w, (mod-1)/n) % mod, inv_fft -> ang^{-1}, root[i] = root[i-1] * ang
  XOR Convolution : set roots[*] = 1, a[j+k] = u+v, a[j+k+i/2] = u-v
   OR Convolution : set roots[*] = 1, a[j+k+i/2] += inv_fft ? -u : u;
  AND Convolution : set roots[*] = 1, a[j+k  ] += inv_fft ? -v : v;
  */
  for(int i=2; i<=N; i<<=1){
    int step = N / i;
    for(int j=0; j<N; j+=i) for(int k=0; k<i/2; k++){
        cpx u = a[j+k], v = a[j+k+i/2] * root[step * k];
        a[j+k] = u+v; a[j+k+i/2] = u-v;
      }
  }
  if(inv_fft) for(int i=0; i<N; i++) a[i] /= N; // skip for AND/OR convolution.
}
vector<ll> multiply(const vector<ll> &_a, const vector<ll> &_b){
  vector<cpx> a(all(_a)), b(all(_b));
  int N = 2; while(N < a.size() + b.size()) N <<= 1;
  a.resize(N); b.resize(N); FFT(a); FFT(b);
  for(int i=0; i<N; i++) a[i] *= b[i];
  vector<ll> ret(N); FFT(a, 1); // NTT : just return a
  for(int i=0; i<N; i++) ret[i] = llround(a[i].real());
  while(ret.size() > 1 && ret.back() == 0) ret.pop_back();
  return ret;
}
// 더 높은 정밀도
vector<ll> multiply_mod(const vector<ll> &a, const vector<ll> &b, const ull mod){
  int N = 2; while(N < a.size() + b.size()) N <<= 1;
  vector<cpx> v1(N), v2(N), r1(N), r2(N);
  for(int i=0; i<a.size(); i++) v1[i] = cpx(a[i] >> 15, a[i] & 32767);
  for(int i=0; i<b.size(); i++) v2[i] = cpx(b[i] >> 15, b[i] & 32767);
  FFT(v1); FFT(v2);
  for(int i=0; i<N; i++){
    int j = i ? N-i : i;
    cpx ans1 = (v1[i] + conj(v1[j])) * cpx(0.5, 0);
    cpx ans2 = (v1[i] - conj(v1[j])) * cpx(0, -0.5);
    cpx ans3 = (v2[i] + conj(v2[j])) * cpx(0.5, 0);
    cpx ans4 = (v2[i] - conj(v2[j])) * cpx(0, -0.5);
    r1[i] = (ans1 * ans3) + (ans1 * ans4) * cpx(0, 1);
    r2[i] = (ans2 * ans3) + (ans2 * ans4) * cpx(0, 1);
  }
  vector<ll> ret(N); FFT(r1, true); FFT(r2, true);
  for(int i=0; i<N; i++){
    ll av = llround(r1[i].real()) % mod;
    ll bv = ( llround(r1[i].imag()) + llround(r2[i].real()) ) % mod;
    ll cv = llround(r2[i].imag()) % mod;
    ret[i] = (av << 30) + (bv << 15) + cv;
    ret[i] %= mod; ret[i] += mod; ret[i] %= mod;
  }
  while(ret.size() > 1 && ret.back() == 0) ret.pop_back();
  return ret;
}
```

4.12 뫼비우스 함수

```
// ||| μ(n) 누적합: M(n)=Σ_{i≤n} μ(i) |||
unordered_map<ll,ll> memoM;
ll getM(ll n){
    if(n<2) return n;
    if(auto it=memoM.find(n); it!=memoM.end()) return it->second;
    ll res = 1;
    for(ll i=2, j; i<=n; i=j+1){
        ll q = n/i;
        j = n/q;
        res = (res - (j-i+1) * (getM(q)%MOD) % MOD + MOD) % MOD;  // divide by g(1)=1 for ϕ
        case
    }
    return memoM[n] = res;
}

// ||| S(N)=Σ_{d≤N} ϕ(d) 재귀 | ϕ*1 = id  ⇒  s_{ϕ*1}(N)=N(N+1)/2
unordered_map<ll,ll> memo0;
ll getS0(ll n){
    if(n<=TH) return pref0[n];  // 아래 전처리 참조
    if(auto it=memo0.find(n); it!=memo0.end()) return it->second;
    ll res = mul(n%MOD, (n+1)%MOD) * inv2 % MOD;  // N(N+1)/2
    // 빼기 Σ_{d=2..n} S0(n/d)*1
    for(ll i=2, j; i<=n; i=j+1){
        ll q = n/i;
        j = n/q;
        ll cnt = (j - i + 1) % MOD;
        res = (res - cnt * getS0(q) % MOD + MOD) % MOD;
    }
    return memo0[n] = res;
}

// ||| S₁(N)=Σ d·ϕ(d) 재귀 | (nϕ)*n = n² ⇒ s_{f*g}=Σ i²
unordered_map<ll,ll> memo1;
ll getS1(ll n){
    if(n<=TH) return pref1[n];
    if(auto it=memo1.find(n); it!=memo1.end()) return it->second;
    // Σ_{i=1..n} i² = n(n+1)(2n+1)/6
    ll res = mul( mul(n%MOD,(n+1)%MOD), ((2*n+1)%MOD) ) * inv6 % MOD;
    // 빼기 Σ_{d=2..n} S1(n/d)*d
```

```
    for(ll i=2, j; i<=n; i=j+1){
        ll q = n/i;
        j = n/q;
        // Σ_{d=i..j} d = (i+j)*(j-i+1)/2
        ll sumd = mul( mul((i+j)%MOD, (j-i+1)%MOD), inv2 );
        res = (res - sumd * getS1(q) % MOD + MOD) % MOD;
    }
    return memo1[n] = res;
}

// ||| S₂(N)=Σ d²·ϕ(d) 재귀 | (n²ϕ)*n² = n³ ⇒ s_{f*g}=Σ i³
unordered_map<ll,ll> memo2;
ll getS2(ll n){
    if(n<=TH) return pref2[n];
    if(auto it=memo2.find(n); it!=memo2.end()) return it->second;
    // Σ i³ = [n(n+1)/2]²
    ll t = mul( mul(n%MOD,(n+1)%MOD), inv2 );
    ll res = mul(t, t);
    // 빼기 Σ_{d=2..n} S2(n/d)*d²
    for(ll i=2, j; i<=n; i=j+1){
        ll q = n/i;
        j = n/q;
        // Σ_{d=i..j} d² = sumSq(j)-sumSq(i-1)
        ll sj = mul( mul(mul(j%MOD,(j+1)%MOD),((2*j+1)%MOD)), inv6 );
        ll si = mul( mul(mul(((i-1)%MOD+MOD)%MOD, i%MOD), ((2*(i-1)+1)%MOD)), inv6 );
        ll sumd2 = sub(sj, si);
        res = (res - sumd2 * getS2(q) % MOD + MOD) % MOD;
    }
    return memo2[n] = res;
}

    TH = min<ll>(D, pow((long double)D, 2.0L/3.0L) + 1);
    pref0.assign(TH+1,0);
    pref1.assign(TH+1,0);
    pref2.assign(TH+1,0);

    // { 선형체로 μ와 ϕ 동시 전처리 {
    vector<int> primes;
    vector<bool> is_comp(TH+1,false);
    vector<int> phi(TH+1,0);
    phi[1]=1;
    for(int i=2;i<=TH;i++){
        if(!is_comp[i]){
            primes.push_back(i);
            phi[i]=i-1;
        }
        for(int p:primes){
            ll x=1LL*p*i;
            if(x>TH) break;
            is_comp[x]=true;
            if(i%p==0){
                phi[x]=phi[i]*p;
                break;
            }else{
                phi[x]=phi[i]*(p-1);
            }
        }
    }

    // { pref sums of ϕ, kϕ, k²ϕ {
    inv2 = (MOD+1)/2;
    inv6 = modpow(6);
    for(int i=1;i<=TH;i++){
        pref0[i] = add(pref0[i-1], phi[i]);
        pref1[i] = add(pref1[i-1], mul(i, phi[i]));
        pref2[i] = add(pref2[i-1], mul(mul(i,i), phi[i]));
    }
```

5 문자열

5.1 문자열 해싱(Rolling Hash) + 예제

Usage: 1-based 닫힌 구간. 길이가 같은 문자열끼리 비교하며, 해시 충돌 가능성이 있다.
Time Complexity: build: $O(N)$, get: $O(1)$

```
// 전처리 O(N), 부분 문자열의 해시값을 O(1)에 구함
// Hashing<917, 998244353> H;
// H.build("ABCDABCD");
// assert(H.get(1, 4) == H.get(5, 8));
// 주의: get 함수의 인자는 1-based 닫힌 구간
// 주의: M은 10억 근처의 소수, P는 M과 서로소

// 1e5+3, 1e5+13, 131'071, 524'287, 1'299'709, 1'301'021
// 1e9-63, 1e9+7, 1e9+9, 1e9+103
template<long long P, long long M> struct Hashing {
    vector<long long> h, p;
    void build(const string &s){
        int n = s.size();
        h = p = vector<long long>(n+1); p[0] = 1;
        for(int i=1; i<=n; i++) h[i] = (h[i-1] * P + s[i-1]) % M;
        for(int i=1; i<=n; i++) p[i] = p[i-1] * P % M;
    }
    long long get(int s, int e) const {
        long long res = (h[e] - h[s-1] * p[e-s+1]) % M;
        return res >= 0 ? res : res + M;
    }
};

// 위 Hashing 정의와 <vector>, <string>, <cassert> 필요.
// h[i] = (h[i-1] * P + s[i-1]) % M
// hash(l, r) = (h[r] - h[l-1] * P^(r-l+1)) mod M
// 앞부분을 빼므로 시작 위치가 달라도 같은 길이끼리 직접 비교 가능.
void rolling_hash_example() {
    string s = "abracadabra", pattern = "abra";
    // 두 모듈러를 함께 비교하여 충돌 가능성을 줄인다.
    Hashing<911382323, 1000000007> a, pa;
    Hashing<911382323, 1000000009> b, pb;
    a.build(s); b.build(s);
    pa.build(pattern); pb.build(pattern);
    assert(a.get(1, 4) == a.get(8, 11));
```

```
    int n = s.size(), m = pattern.size(); // 이 예제는 m >= 1
    vector<int> positions;
    for (int l = 1; l + m - 1 <= n; ++l) {
        int r = l + m - 1;
        if (a.get(l, r) == pa.get(1, m) &&
            b.get(l, r) == pb.get(1, m)) positions.push_back(l);
    }
    assert((positions == vector<int>{1, 8})); // 1-based 시작 위치
    // 전체 O(n+m). 이중 해시도 충돌 가능성은 0이 아니다.
    // 정확한 매칭 보장이 필요하면 KMP/Z 사용.
}
```

5.2 문자열 매칭 - KMP

Time Complexity: GetFail: $O(|P|)$, $O(|S|+|P|)$

```
// s에서 p가 등장하는 위치 반환
// KMP("ABABCAB", "AB") = {0, 2, 5}
// KMP("AAAA", "AA") = {0, 1, 2}

vector<int> GetFail(const string &p){
    int n = p.size();
    vector<int> fail(n);
    for(int i=1, j=0; i<n; i++){
        while(j && p[i] != p[j]) j = fail[j-1];
        if(p[i] == p[j]) fail[i] = ++j;
    }
    return fail;
}

vector<int> KMP(const string &s, const string &p){
    int n = s.size(), m = p.size();
    vector<int> fail = GetFail(p), ret;
    for(int i=0, j=0; i<s.size(); i++){
        while(j && s[i] != p[j]) j = fail[j-1];
        if(s[i] == p[j]){
            if(j + 1 == m) ret.push_back(i-m+1), j = fail[j];
            else j++;
        }
    }
    return ret;
}
```

5.3 가장 긴 팰린드롬 부분 분자열 - Manacher

Time Complexity: $O(N)$

```
// 각 문자를 중심으로 하는 최장 팰린드롬의 반경을 반환
// Manacher("abaaba") = {0,1,0,3,0,1,6,1,0,3,0,1,0}
// # a # b # a # a # b # a #
// 0 1 0 3 0 1 6 1 0 3 0 1 0
vector<int> Manacher(const string &inp){
    int n = inp.size() * 2 + 1;
    vector<int> ret(n);
    string s = "#";
    for(auto i : inp) s += i, s += "#";
    for(int i=0, p=-1, r=-1; i<n; i++){
        ret[i] = i <= r ? min(r-i, ret[2*p-i]) : 0;
        while(i-ret[i]-1 >= 0 && i+ret[i]+1 < n && s[i-ret[i]-1] == s[i+ret[i]+1])
        ret[i]++;
        if(i+ret[i] > r) r = i+ret[i], p = i;
    }
    return ret;
}
```

5.4 문자열 매칭 - Z

Time Complexity: $O(N)$

```
// Z[i] = LongestCommonPrefix(S[0:N], S[i:N])
//      = S[0:N]과 S[i:N]이 앞에서부터 몇 글자 겹치는지
vector<int> Z(const string &s){
    int n = s.size();
    vector<int> z(n);
    z[0] = n;
    for(int i=1, l=0, r=0; i<n; i++){
        if(i < r) z[i] = min(r-i-1, z[i-l]);
        while(i+z[i] < n && s[i+z[i]] == s[z[i]]) z[i]++;
        if(i+z[i] > r) r = i+z[i], l = i;
    }
    return z;
}
```

5.5 접미사 배열

Time Complexity: $O(N \log N)$, suffix array는 $O(N \log^2 N)$에 다음의 관찰을 기반으로 구할 수 있다.

- 모든 접미사의 k번째 문자열까지를 기준으로 정렬되어 있다고 가정하자. - 그러면 (i번째 접미사의 결정된 순서, $i+k$번째 접미사의 결정된 순서)를 쌍으로 해 정렬하면 $i+k$번째 접미사의 결정된 순서에 추가 k자리의 순서가 나와 있으므로 전체 $2k$자리를 정렬된 상태로 둘 수 있다.

즉 k를 1부터 증가시키면서 n보다 커질때까지 하면 된다.

```
// LCP는 1-based
pair<vector<int>, vector<int>> SuffixArray(const string &s){ // O(N log N)
  int n = s.size(), m = max(n, 256);
  vector<int> sa(n), lcp(n), pos(n), tmp(n), cnt(m);
  auto counting_sort = [&](){
    fill(cnt.begin(), cnt.end(), 0);
    for(int i=0; i<n; i++) cnt[pos[i]]++;
    partial_sum(cnt.begin(), cnt.end(), cnt.begin());
    for(int i=n-1; i>=0; i--) sa[--cnt[pos[tmp[i]]]] = tmp[i];
  };
  for(int i=0; i<n; i++) sa[i] = i, pos[i] = s[i], tmp[i] = i;
  counting_sort();
  for(int k=1; ; k<<=1){
    int p = 0;
    for(int i=n-k; i<n; i++) tmp[p++] = i;
    for(int i=0; i<n; i++) if(sa[i] >= k) tmp[p++] = sa[i] - k;
    counting_sort();
    tmp[sa[0]] = 0;
    for(int i=1; i<n; i++){
```

```
      tmp[sa[i]] = tmp[sa[i-1]];
      if(sa[i-1]+k < n && sa[i]+k < n && pos[sa[i-1]] == pos[sa[i]] && pos[sa[i-1]+k] ==
      pos[sa[i]+k]) continue;
      tmp[sa[i]] += 1;
    }
    swap(pos, tmp); if(pos[sa.back()] + 1 == n) break;
  }
  for(int i=0, j=0; i<n; i++, j=max(j-1,0)){
    if(pos[i] == 0) continue;
    while(sa[pos[i]-1]+j < n && sa[pos[i]]+j < n && s[sa[pos[i]-1]+j] == s[sa[pos[i]]+j])
    j++;
    lcp[pos[i]] = j;
  }
  return {sa, lcp};
}
```

6 계산 기하

6.1 2차원 계산 기하 템플릿 + CCW

```
#include <bits/stdc++.h>
#define x first
#define y second
using namespace std;
using ll = long long;
using Point = pair<ll, ll>;

Point operator + (Point p1, Point p2){ return {p1.x + p2.x, p1.y + p2.y}; }
Point operator - (Point p1, Point p2){ return {p1.x - p2.x, p1.y - p2.y}; }
ll operator * (Point p1, Point p2){ return p1.x * p2.x + p1.y * p2.y; } // 내적
ll operator / (Point p1, Point p2){ return p1.x * p2.y - p2.x * p1.y; } // 외적
int Sign(ll v){ return (v > 0) - (v < 0); } // 양수면 +1, 음수면 -1, 0이면 0 반환
ll Dist(Point p1, Point p2){ return (p2 - p1) * (p2 - p1); } // 두 점 거리 제곱
ll SignedArea(Point p1, Point p2, Point p3){ return (p2 - p1) / (p3 - p1); }
int CCW(Point p1, Point p2, Point p3){ return Sign(SignedArea(p1, p2, p3)); }
```

6.2 360도 각도 정렬

```
/* y축
   ↑
3  2 1
4 -1 0 → x축
5  6 7
원점 기준 각도 정렬
x축 위에 있는 점이 가장 먼저, 그리고 반시계 방향으로
*/
int QuadrantID(const Point p){
  static int arr[9] = { 5, 4, 3, 6, -1, 2, 7, 0, 1 };
  return arr[Sign(p.x)*3+Sign(p.y)+4];
}
sort(points.begin(), points.end(), [&](auto p1, auto p2){
  if(QuadrantID(p1) != QuadrantID(p2)) return QuadrantID(p1) < QuadrantID(p2);
  else return p1 / p2 > 0; // 반시계
});
```

6.3 다각형 넓이

```
// 다각형의 넓이의 2배를 반환, 항상 정수, O(N)
ll PolygonArea(const vector<Point> &v){
    ll res = 0;
    for(int i=0; i<v.size(); i++) res += v[i] / v[(i+1)%v.size()];
    return abs(res);
}
```

6.4 선분 교차 판정

```
// 선분 교차 - 선분 ab와 선분 cd가 만나면 true
bool Cross(Point s1, Point e1, Point s2, Point e2){
    int ab = CCW(s1, e1, s2) * CCW(s1, e1, e2);
    int cd = CCW(s2, e2, s1) * CCW(s2, e2, e1);
    if(ab == 0 && cd == 0){
        if(s1 > e1) swap(s1, e1);
        if(s2 > e2) swap(s2, e2);
        return !(e1 < s2 || e2 < s1);
    }
    return ab <= 0 && cd <= 0;
}
// 교차하지 않으면 0
// 교점이 무한히 많으면 -1
// 교점이 1개면 1 반환하고 res에 교점 저장
int Cross(Point s1, Point e1, Point s2, Point e2, pair<double, double> &res){
    if(!Cross(s1, e1, s2, e2)) return 0;
    ll det = (e1 - s1) / (e2 - s2);
    if(!det){
        if(s1 > e1) swap(s1, e1);
        if(s2 > e2) swap(s2, e2);
        if(e1 == s2){ res = s2; return 1; }
        if(e2 == s1){ res = s1; return 1; }
        return -1;
    }
    res.x = s1.x + (e1.x - s1.x) * ((s2 - s1) / (e2 - s2) * 1.0 / det);
    res.y = s1.y + (e1.y - s1.y) * ((s2 - s1) / (e2 - s2) * 1.0 / det);
    return 1;
}
```

6.5 다각형 내부 판별

```
// 다각형 내부 또는 경계 위에 p가 있으면 true, O(N)
bool PointInPolygon(const vector<Point> &v, Point p){
    int n = v.size(), cnt = 0;
    Point p2(p.x+1, 1'000'000'000 + 1); // 좌표 범위보다 큰 수
    for(int i=0; i<n; i++){
        int j = i + 1 < n ? i + 1 : 0;
        if(min(v[i],v[j]) <= p && p <= max(v[i],v[j]) && CCW(v[i], v[j], p) == 0) return
        true;
        if(SegmentIntersection(v[i], v[j], p, p2)) cnt++;
    }
    return cnt % 2 == 1;
}
```

6.6 볼록 껍질 - Graham Scan

```
// 모든 점을 포함하는 가장 작은 볼록 다각형, O(N log N)
vector<Point> ConvexHull(vector<Point> points){
    if(points.size() <= 1) return points;
    swap(points[0], *min_element(points.begin(), points.end()));
    sort(points.begin()+1, points.end(), [&](auto a, auto b){
        int dir = CCW(points[0], a, b);
        if(dir != 0) return dir > 0;
        return Dist(points[0], a) < Dist(points[0], b);
    });
    vector<Point> hull;
    for(auto p : points){
        while(hull.size() >= 2 && CCW(hull[hull.size()-2], hull.back(), p) <= 0)
        hull.pop_back();
        hull.push_back(p);
    }
    return hull;
}
```

6.7 가장 먼 두 점 - Rotating Calipers

```
// 가장 먼 두 점을 구하는 함수, O(N)
// 주의: hull은 반시계 방향으로 정렬된 볼록 다각형이어야 함
pair<Point, Point> Calipers(vector<Point> hull){
    int n = hull.size(); ll mx = 0; Point a, b;
    for(int i=0, j=0; i<n; i++){
        while(j + 1 < n && (hull[i+1] - hull[i]) / (hull[j+1] - hull[j]) >= 0){
            ll now = Dist(hull[i], hull[j]);
            if(now > mx) mx = now, a = hull[i], b = hull[j];
            j++;
        }
        ll now = Dist(hull[i], hull[j]);
        if(now > mx) mx = now, a = hull[i], b = hull[j];
    }
    return {a, b};
}
```

6.8 볼록 다각형 내부 판별

```
// 다각형 내부 또는 경계 위에 p가 있으면 true, O(log N)
// 주의: v는 반시계 방향으로 정렬된 볼록 다각형이어야 함
bool PointInConvexPolygon(const vector<Point> &v, const Point &pt){
    if(CCW(v[0], v[1], pt) < 0) return false; int l = 1, r = v.size() - 1;
    while(l < r){
        int m = l + r + 1 >> 1;
        if(CCW(v[0], v[m], pt) >= 0) l = m; else r = m - 1;
    }
    if(l == v.size() - 1) return CCW(v[0], v.back(), pt) == 0 && v[0] <= pt && pt <=
    v.back();
    return CCW(v[0], v[l], pt) >= 0 && CCW(v[l], v[l+1], pt) >= 0 && CCW(v[l+1], v[0], pt)
    >= 0;
}
```

7 기타

7.1 가장 긴 증가하는 부분 수열(LIS)

Time Complexity: $O(N \log N)$

```
// LIS({10, 20, 10, 30, 20, 50}) = {10, 20, 30, 50}
vector<int> LIS(const vector<int> &v){
  int n = v.size();
  vector<int> pos(n), lis, res;
  for(int i=0; i<n; i++){
    if(lis.empty() || lis.back() < v[i]){
      lis.push_back(v[i]); pos[i] = lis.size();
    }
    else{
      int idx = lower_bound(lis.begin(), lis.end(), v[i]) - lis.begin();
      lis[idx] = v[i]; pos[i] = idx + 1;
    }
  }
  int len = lis.size();
  for(int i=n-1; i>=0; i--) if(pos[i] == len) res.push_back(v[i]), len--;
  reverse(res.begin(), res.end());
  return res;
}
```

7.2 이분 탐색

```
// 주의: 구간 범위에 음수가 들어가면 / 2 말고 >> 1로 해야 함

// 1 1 1 ... 1 1 0 0 ... 0 꼴일 때 마지막 1의 위치
int l = 1, r = n;
while(l < r){
    int m = (l + r + 1) / 2;
    if(Check(m)) l = m;
    else r = m - 1;
}

// 0 0 0 ... 0 0 1 1 ... 1 꼴일 때 첫 번째 1의 위치
int l = 1, r = n;
while(l < r){
    int m = (l + r) / 2;
    if(Check(m)) r = m;
    else l = m + 1;
}
```

7.3 삼분 탐색

```
while(s + 3 <= e){ // get minimum / when multiple answer, find minimum `s`
  T l = (s + s + e) / 3, r = (s + e + e) / 3;
  if(Check(l) > Check(r)) s = l; else e = r;
}
T mn = INF, idx = s;
for(T i=s; i<=e; i++) if(T now = Check(i); now < mn) mn = now, idx = i;
```

7.4 C++ 랜덤, GCC 확장, 비트마스킹 트릭

```
mt19937 rd((unsigned)chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<int> rnd_int(l, r); // rnd_int(rd)
uniform_real_distribution<double> rnd_real(0, 1); // rnd_real(rd)
/////
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/rope>
using namespace __gnu_pbds; //ordered_set : find_by_order(order), order_of_key(key)
using namespace __gnu_cxx; //crope : append(str), substr(s, e), at(idx)
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
/////
int __builtin_clz(int x);// number of leading zero
int __builtin_ctz(int x);// number of trailing zero
int __builtin_popcount(int x);// number of 1-bits in x
lsb(n): (n & -n); // last bit (smallest)
floor(log2(n)): 31 - __builtin_clz(n | 1);
floor(log2(n)): 63 - __builtin_clzll(n | 1);
long long next_perm(long long v){
  long long t = v | (v-1);
  return (t + 1) | (((~t & -~t) - 1) >> (__builtin_ctz(v) + 1));
}
int frq(int n, int i) { // # of digit i in [1, n]
  int j, r = 0;
  for (j = 1; j <= n; j *= 10) if (n / j / 10 >= !i) r += (n / 10 / j - !i) * j + (n / j %
  10 > i ? j : n / j % 10 == i ? n % j + 1 : 0);
  return r;
}
```

7.5 C++ 입력: EOF, 한 줄 입력 / 출력 precision

Usage: 각 함수는 독립적인 사용 예제.

```
// 필요: <iostream>, <iomanip>, <string>, <sstream>, <limits>
// using namespace std; 각 함수는 별개의 입력 상황에서 사용.
// main 시작: ios::sync_with_stdio(false); cin.tie(nullptr);
// 동기화를 끈 뒤에는 scanf/printf와 cin/cout을 섞지 않는다.

void read_until_eof() { // 입력 개수 없이 EOF까지 정수 입력
    long long x;
    while (cin >> x) cout << x << '\n';
    // 두 개씩: while (cin >> a >> b) { ... }
    // 잘못된 형식도 반복 종료. while (!cin.eof())는 사용하지 않는다.
}
void read_lines() { // 공백/빈 줄을 포함하여 한 줄씩, EOF까지
    string line;
    while (getline(cin, line)) {
        // getline은 개행 문자를 소비하지만 line에는 넣지 않는다.
        cout << line << '\n';
    }
}
void read_after_number() {
    int n;
    if (!(cin >> n)) return;
    // >>가 남긴 개행을 포함해 현재 줄의 나머지를 버린 뒤 다음 줄 입력
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    string line;
    if (!getline(cin, line)) return;
    // 한 줄에 있는 정수만 읽기 (개수 미지정)
    istringstream iss(line);
    int x;
    while (iss >> x) cout << x << '\n';
    // getline(cin >> ws, line)은 선행 공백과 빈 줄까지 모두 건너뜀.
    // 공백/빈 줄 보존이 필요하면 위의 ignore + getline 사용.
}
void precision_example() {
    double x = 123.456789;
    cout << fixed << setprecision(3) << x << '\n'; // 123.457 (소수 3자리)
    cout << defaultfloat << setprecision(5) << x << '\n'; // 123.46 (유효 5자리)
    cout << scientific << setprecision(2) << x << '\n'; // 1.23e+02
    // 설정은 이후 출력에도 유지. 기본 상태로 복원:
    cout << defaultfloat << setprecision(6);
}
```

7.6 빠른 입력(Fast Input from STDIN)

```
namespace io {
  const signed IS=1<<20;
  char I[IS+1],*J=I;
  inline void daer(){if(J>=I+IS-64){
    char*p=I;do*p++=*J++;
    while(J!=I+IS);p[read(0,p,I+IS-p)]=0;J=I;}}
  template<int N=10,typename T=int>inline T getu(){
    daer();T x=0;int k=0;do x=x*10+*J-'0';
    while(*++J>='0'&&++k<N);++J;return x;}
  template<int N=10,typename T=int>inline T geti(){
    daer();bool e=*J=='-';J+=e;return(e?-1:1)*getu<N,T>();}
  struct f{f(){I[read(0,I,IS)]=0;}}flu;
};
```

7.7 구간별 약수 최대 개수, 최대 소수

```
< 10^k          number     divisors   2 3 5 71113171923293137
-------------------------------------------------------------
1                    6            4   1 1
2                   60           12   2 1 1
3                  840           32   3 1 1 1
4                 7560           64   3 3 1 1
5                83160          128   3 3 1 1 1
6               720720          240   4 2 1 1 1 1
7              8648640          448   6 3 1 1 1 1
8             73513440          768   5 3 1 1 1 1 1
9            735134400         1344   6 3 2 1 1 1 1
10          6983776800         2304   5 3 2 1 1 1 1 1
11         97772875200         4032   6 3 2 2 1 1 1 1
12        963761198400         6720   6 4 2 1 1 1 1 1 1
13       9316358251200        10752   6 3 2 1 1 1 1 1 1 1
14      97821761637600        17280   5 4 2 2 1 1 1 1 1 1
```

15	866421317361600	26880	6 4 2 1 1 1 1 1 1 1 1
16	8086598962041600	41472	8 3 2 2 1 1 1 1 1 1 1
17	74801040398884800	64512	6 3 2 2 1 1 1 1 1 1 1 1 1
18	897612484786617600	103680	8 4 2 2 1 1 1 1 1 1 1 1

< 10^k	prime	# of prime	< 10^k	prime
1	7	4	10	9999999967
2	97	25	11	99999999977
3	997	168	12	999999999989
4	9973	1229	13	9999999999971
5	99991	9592	14	99999999999973
6	999983	78498	15	999999999999989
7	9999991	664579	16	9999999999999937
8	99999989	5761455	17	99999999999999997
9	999999937	50847534	18	999999999999999989

7.8 카탈란 수, 심슨 적분, 그런디 정리, 픽의 정리, 페르마 포인트, 오일러 정리

- 카탈란 수
  1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012,742900
  $C_n = binomial(n*2, n)/(n+1)$;
  - 길이가 2n인 올바른 괄호 수식의 수
  - n + 1개의 리프를 가진 풀 바이너리 트리의 수
  - n + 2각형을 n개의 삼각형으로 나누는 방법의 수
- Simpson 공식 (적분) : Simpson 공식, $S_n(f) = \frac{h}{3}[f(x_0) + f(x_n) + 4\sum f(x_{2i+1}) + 2\sum f(x_{2i})]$
  - $M = \max|f^4(x)|$이라고 하면 오차 범위는 최대 $E_n \le \frac{M(b-a)}{180}h^4$
- 알고리즘 게임
  - Nim Game의 해법 : 각 더미의 돌의 개수를 모두 XOR했을 때 0 이 아니면 첫번째, 0 이면 두번째 플레이어가 승리.
  - Grundy Number : 어떤 상황의 Grundy Number는, 가능한 다음 상황들의 Grundy Number를 모두 모은 다음, 그 집합에 포함 되지 않는 가장 작은 수가 현재 state의 Grundy Number가 된다. 만약 다음 state가 독립된 여러개의 state들로 나뉠 경우, 각각의 state의 Grundy Number의 XOR 합을 생각한다.
  - Subtraction Game : 한 번에 k 개까지의 돌만 가져갈 수 있는 경우, 각 더미의 돌의 개수를 k + 1로 나눈 나머지를 XOR 합하여 판단한다.
  - Index-k Nim : 한 번에 최대 k개의 더미를 골라 각각의 더미에서 아무렇게나 돌을 제거할 수 있을 때, 각 binary digit에 대하여 합을 k + 1로 나눈 나머지를 계산한다. 만약 이 나머지가 모든 digit에 대하여 0이라면 두번째, 하나라도 0이 아니라면 첫번째 플레이어가 승리.
  - Misere Nim : 모든 돌 무더기가 1이면 N이 홀수일 때 후공 승, 그렇지 않은 경우 XOR 합 0이면 후공 승
- Pick's Theorem
  격자점으로 구성된 simple polygon이 주어짐. I 는 polygon 내부의 격자점 수, B 는 polygon 선분 위 격자점 수, A는 polygon의 넓이라고 할 때, 다음과 같은 식이 성립한다. $A = I + B/2 - 1$

```
// number of (x, y) : (0 <= x < n && 0 < y <= k/d x + b/d)
ll count_solve(ll n, ll k, ll b, ll d) { // argument should be positive
  if (k == 0) {
    return (b / d) * n;
  }
  if (k >= d || b >= d) {
    return ((k / d) * (n - 1) + 2 * (b / d)) * n / 2 + count_solve(n, k % d, b % d, d);
  }
  return count_solve((k * n + b) / d, d, (k * n + b) % d, k);
}
```

- 페르마 포인트 : 삼각형의 세 꼭짓점으로부터 거리의 합이 최소가 되는 점
  $2\pi/3$ 보다 큰 각이 있으면 그 점이 페르마 포인트, 그렇지 않으면 각 변마다 정삼각형 그린 다음, 정삼각형의 끝점에서 반대쪽 삼각형의 꼭짓점으로 연결한 선분의 교점
  $2\pi/3$ 보다 큰 각이 없으면 거리의 합은 $\sqrt{(a^2+b^2+c^2+4\sqrt{3}S)/2}$, S는 넓이
- 오일러 정리: 서로소인 두 정수 a, n에 대해 $a^{\phi(n)} \equiv 1 \pmod n$
  모든 정수에 대해 $a^n \equiv a^{n-\phi(n)} \pmod n$
  $m \ge log_2 n$이면 $a^m \equiv a^{m\%\phi(n)+\phi(n)} \pmod n$
- $g^0 + g^1 + g^2 + \cdots g^{p-2} \equiv -1 \pmod p$ iff $g = 1$, otherwise 0.

7.9 경우의 수 - 포함 배제, 스털링 수, 벨 수

- 공 구별 X, 상자 구별 O, 전사함수 : 포함배제 $\sum_{i=1}^{k}(-1)^{k-i} \times kCi \times i^n$
- 공 구별 O, 상자 구별 X, 전사함수 : 제 2종 스털링 수 $S(n,k) = k \times S(n-1,k) + S(n-1,k-1)$
  포함배제하면 $O(K \log N)$, $S(n,k) = 1/k! \times \sum_{i=1}^{k}(-1)^{k-i} \times kCi \times i^n$
- 공 구별 O, 상자 구별 X, 제약없음 : 벨 수 $B(n,k) = \sum_{i=0}^{k} S(n,i)$ 몇 개의 상자를 버릴지 다 돌아보기
  수식 정리하면 $O(\min(N,K)\log N)$에 됨. $B(n,n) = \sum_{i=0}^{n-1}(n-1)Ci \times B(i,i)$
  $B(n,k) = \sum_{j=0}^{k} S(n,j) = \sum_{j=0}^{k} 1/j! \sum_{i=0}^{j}(-1)^{j-i} jCi \times i^n = \sum_{j=0}^{k}\sum_{i=0}^{j}\frac{(-1)^{j-i}}{i!(j-i)!}i^n$
  $= \sum_{i=0}^{k}\sum_{j=i}^{k}\frac{(-1)^{j-i}}{i!(j-i)!}i^n = \sum_{i=0}^{k}\sum_{j=0}^{k-i}\frac{(-1)^{j}}{i!j!}i^n = \sum_{i=0}^{k}\frac{i^n}{i!}\sum_{j=0}^{k-i}\frac{(-1)^j}{j!}$
- Derangement: $D(n) = (n-1)(D(n-1) + D(n-2))$
- Signed Stirling 1: $S_1(n,k) = (n-1)S_1(n-1,k) + S_1(n-1,k-1)$
- Unsigned Stirling 1: $C_1(n,k) = (n-1)C_1(n-1,k) + C_1(n-1,k-1)$
- Stirling 2: $S_2(n,k) = kS_2(n-1,k) + S_2(n-1,k-1)$
- Stirling 2: $S_2(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$
- Partition: $p(n,k) = p(n-1,k-1) + p(n-k,k)$
- Partition: $p(n) = \sum(-1)^k p(n-k(3k-1)/2)$
- Bell: $B(n) = \sum_{k=1}^{n}\binom{n-1}{k-1}B(n-k)$
- Catalan: $C_n = \frac{1}{n+1}\binom{2n}{n}$
- Catalan: $C_n = \binom{2n}{n} - \binom{2n}{n+1}$
- Catalan: $C_n = \frac{(2n)!}{n!(n+1)!}$
- Catalan: $C_n = \sum C_i C_{n-i}$

7.10 삼각형의 오심 - 외심, 내심, 무게중심, 수심, 방심

변 길이 $a, b, c; p = (a+b+c)/2$
넓이 $A = \sqrt{p(p-a)(p-b)(p-c)}$
외접원 반지름 $R = abc/4A$, 내접원 반지름 $r = A/p$
중선 길이 $m_a = 0.5\sqrt{2b^2+2c^2-a^2}$
각 이등분선 길이 $s_a = \sqrt{bc(1-\frac{a}{b+c}^2)}$
사인 법칙 $\frac{sinA}{a} = 1/2R$, 코사인 법칙 $a^2 = b^2 + c^2 - 2bc\cos A$, 탄젠트 법칙 $\frac{a+b}{a-b} = \frac{\tan(A+B)/2}{\tan(A-B)/2}$
중심 좌표 $(\frac{\alpha x_a+\beta x_b+\gamma x_c}{\alpha+\beta+\gamma}, \frac{\alpha y_a+\beta y_b+\gamma y_c}{\alpha+\beta+\gamma})$

이름	α	β	γ	
외심	$a^2\mathcal{A}$	$b^2\mathcal{B}$	$c^2\mathcal{C}$	$\mathcal{A}=b^2+c^2-a^2$
내심	a	b	c	$\mathcal{B}=a^2+c^2-b^2$
무게중심	1	1	1	$\mathcal{C}=a^2+b^2-c^2$
수심	$\mathcal{BC}$	$\mathcal{CA}$	$\mathcal{AB}$	
방심(A)	$-a$	b	c	

7.11 미적분, 뉴턴 랩슨법

- $(\arcsin x)' = 1/\sqrt{1-x^2}$
- $(\tan x)' = 1+\tan^2 x$
- $\int tanax = -\ln|\cos ax|/a$
- $(\arccos x)' = -1/\sqrt{1-x^2}$
- $(\arctan x)' = 1/(1+x^2)$
- $\int x\sin ax = (\sin ax - ax\cos ax)/a^2$
- Newton: $x_{n+1} = x_n - f(x_n)/f'(x_n)$
- $\oint_C (Ldx+Mdy) = \int\int_D (\frac{\partial M}{\partial x} - \frac{\partial L}{\partial y})dxdy$
- where C is positively oriented, piecewise smooth, simple, closed; D is the region inside C; L and M have continuous partial derivatives in D.

7.12 문제 풀이 체크리스트

- 비슷한 문제를 풀어본 적이 있던가?
- 단순한 방법에서 시작할 수 있을까? (Brute Force)
- 내가 문제를 푸는 과정을 수식화할 수 있을까? (예제를 직접 해결해보면서)
- 문제를 단순화할 수 없을까?
- 그림으로 그려볼 수 있을까?
- 수식으로 표현할 수 있을까?
- 문제를 분해할 수 있을까?
- 뒤에서부터 생각해서 풀 수 있을까?
- 순서를 강제할 수 있을까?
- 특정 형태의 답만을 고려할 수 있을까? (정규화)
- 구간을 통째로 가져간다 : 플로우 + 적당한 자료구조 $(i, i+1, k, 0), (s, e, 1, w), (N, T, k, 0)$
- a = b : a만 움직이기, b만 움직이기, 두 개 동시에 움직이기, 반대로 움직이기
- 말도 안 되는 것들을 한 번은 생각해보기 / "당연하다고 생각한 것" 다시 생각해보기
- 확률 : DP, 이분 탐색(NYPC 2019 Finals C)
- 최대/최소 : 이분 탐색, 그리디(Prefix 고정, Exchange Argument), DP(순서 고정)